

Tarea 1 Manejo de Datos

David Angel Gonzalez Hidalgo

Diego Zaid Reyes Miranda

Luis Daniel Arévalo Esquivel

Parte 1: Investigación

Reglas PEP8:

1. Usa 4 espacios por nivel. Nunca mezcles espacios y tabuladores.
2. Longitud de línea: Máximo 79 caracteres en código y 72 en comentarios/docstrings.
3. Espacios en expresiones: Coloca espacios alrededor de operadores (`a = b + c`), pero evita espacios dentro de paréntesis o corchetes.
4. Imports: Hazlos al inicio del archivo, separados en grupos. Nunca uses `import *`.
5. Nombres:
 - Variables y funciones: `snake_case` (ej. `calculate_total`).
 - Clases: `CamelCase` (ej. `CustomerOrder`).
 - Constantes: mayúsculas con guiones bajos (`MAX_SIZE`).
6. Comentarios: Claros y útiles. Los comentarios en línea van después de `#`, los de bloque encima del código.
7. Docstrings: Usa triple comillas dobles (`"""`) para describir funciones, clases y módulos siguiendo PEP 257.
8. Comas finales: Úsalas en listas largas para facilitar futuras modificaciones.
9. Argumentos de métodos: Siempre usa `self` en métodos de instancia y `cls` en métodos de clase.
10. Codificación de archivos: Usa UTF-8 por defecto para asegurar compatibilidad internacional.

¿Qué es SOLID?:

Las reglas "SOLID" son un conjunto de principios orientados a la creación de programas de manera limpia, ordenada y flexible postulados por Robert C. Martin.

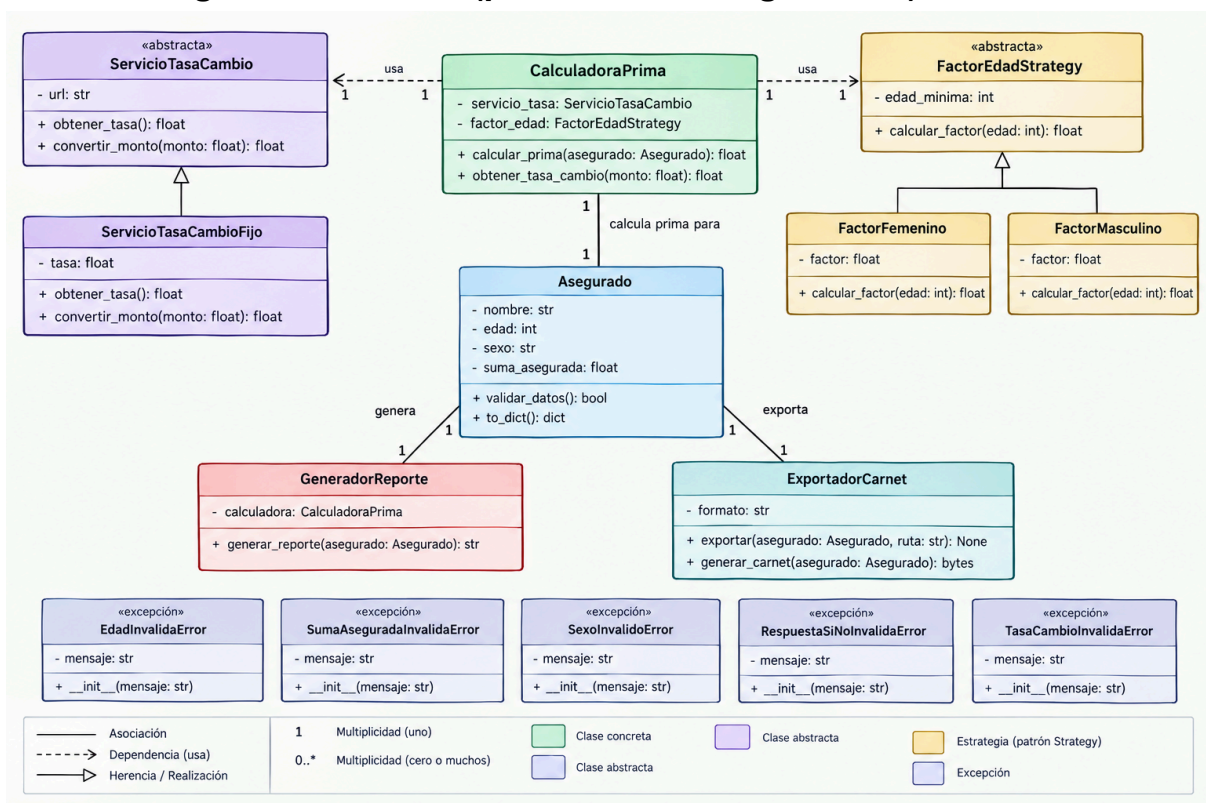
El nombre SOLID deriva del acrónimo formado por el nombre de las 5 reglas que componen este grupo, los cuales son:

- Single Responsibility Principle (Principio de responsabilidad única): Este principio busca evitar el uso de clases que hacen muchas cosas y tienen diferentes razones para cambiar. Esto a través de la asignación de una tarea concreta por clase.
- Open/Closed Principle (Principio de abierto/cerrado): Aquí se propone que las clases usadas deben poder alterar su comportamiento sin afectar el funcionamiento de un

código ya funcional. Esto para evitar modificar un programa ya existente cada vez que se le agregue algo a una clase.

- Liskov Substitution Principle (Principio de sustitución de Liskov): Para casos donde una subclase no pueden comportarse concretamente como sus clases padre. Este punto propone que una clase hija puede sustituir algunos objetos de la clase padre sin alterar el comportamiento del programa.
- Interface Segregation Principle (Principio de segregación de interfaces): Este principio nos habla del uso de interfaces pequeñas y específicas en lugar de interfaces grandes que obligan a las clases del programa a implementar métodos que no necesitan.
- Dependency inversion Principle (Principio de inversión de dependencias): Para evitar ligar el código a una clase concreta que dificulte una variación en los métodos de las subclases y demás partes del código, este principio propone la dependencia a abstracciones para las características necesarias en cada caso.

Parte 2: Diagrama de clases (problema de aseguradora).



Fuentes:

Python Software Foundation. (2001). PEP 8 – Style Guide for Python Code. Recuperado de <https://peps.python.org/pep-0008/>

Ryware. (2026, julio 18). Principios SOLID: guía práctica para el diseño de sistemas.
Recuperado de
<https://ryware.com/blog/principios-solid-guia-practica>